

Challenge Report

Challenge Report – Ticket Management System

Table of Contents

1. [Project Overview](#1-project-overview)
2. [Architecture & Design Decisions](#2-architecture--design-decisions)
3. [Infrastructure Setup](#3-infrastructure-setup)
4. [Backend Setup & Commands](#4-backend-setup--commands)
5. [Database Setup & Migrations](#5-database-setup--migrations)
6. [Data Preparation & Seeding](#6-data-preparation--seeding)
7. [Frontend Setup & Commands](#7-frontend-setup--commands)
8. [Testing Strategy](#8-testing-strategy)
9. [API Endpoints Reference](#9-api-endpoints-reference)
10. [Environment Variables](#10-environment-variables)
11. [Key Technical Decisions](#11-key-technical-decisions)

1. Project Overview

A full-stack ticket/case management system built with **Django REST Framework** (backend) and **SvelteKit** (frontend). The application supports role-based access control (ADMIN, MANAGER, AGENT) with asynchronous email notifications via Celery, Redis caching, RabbitMQ event publishing, and Sentry error monitoring.

Core Features

- JWT-based authentication with refresh token rotation
- Ticket CRUD with role-based permissions
- Comments on tickets
- Audit history tracking (field-level changes)
- In-app notifications with async email delivery
- Dashboard statistics with Redis caching
- RabbitMQ event bus for domain events
- Sentry integration (backend + frontend)
- Form validation with field-level error handling
- Modern UI with indigo design system

2. Architecture & Design Decisions

Backend Architecture: Service → Selector → Serializer → View

```

View (HTTP layer, permissions, response format)

■■■ Serializer (validation & data transformation)

■■■ Selector (query logic, filtering)

■■■ Service (database operations & side effects)

```

This layered architecture was chosen to:

- Keep views thin (only HTTP concerns)
- Centralize business logic in services (notifications, event publishing, cache invalidation)
- Make code testable at each layer independently

Key Decisions

Decision	Rationale
----------	-----------

JWT over session auth	Stateless API suitable for SPA frontend
-----------------------	---

Celery for email	Non-blocking email delivery, auto-retry on failure
------------------	--

Redis as Celery broker	Simpler than RabbitMQ for task queuing;
------------------------	---

RabbitMQ kept for domain events	
---------------------------------	--

RabbitMQ for events	Decoupled event publishing; consumers can subscribe independently
---------------------	---

UUID primary keys	Prevents ID enumeration, safer for public-facing IDs
-------------------	--

Django built-in RedisCache	No extra dependency; `django-redis` installed but `RedisCache` used
----------------------------	---

`CELERY_TASK_ALWAYS_EAGER` env var	Allows running tests synchronously without a worker
------------------------------------	---

3. Infrastructure Setup

Services (docker-compose.yml)

Service	Image	Port	Purpose
db	postgres:16-alpine	5432	Primary database
redis	redis:7-alpine	6379	Cache + Celery broker/backend
rabbitmq	rabbitmq:3.13-management-alpine	5672, 15672	Event bus
backend	Custom (Dockerfile)	8000	Django API server
celery_worker	Custom (Dockerfile)	-	Background task processing
frontend	Custom (Dockerfile)	5173	SvelteKit dev server

Start all services

```
```bash
docker compose up -d
or with Podman:
podman-compose up -d
```
```

Start individual services

```
```bash
docker compose up -d db redis rabbitmq # infrastructure only
docker compose up -d backend # Django API
docker compose up -d celery_worker # Celery worker
docker compose up -d frontend # SvelteKit dev server
```
```

Stop all services

```
```bash
docker compose down
```
```

4. Backend Setup & Commands

Local setup (without Docker)

```
```bash
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # then edit with your values
```
```

Running the dev server

```
```bash
python manage.py runserver
API available at http://localhost:8000/api/
Swagger docs at http://localhost:8000/api/docs/
Django admin at http://localhost:8000/admin/
```
```

Running the Celery worker

```
```bash
celery -A core worker --loglevel=info
```
```

Linting

```
```bash
ruff check .
ruff format --check .
```
```

Auto-format

```
```bash
ruff format .
```
```

```
```
```

```
Django management commands
```

```
```bash
```

```
# Create a superuser
```

```
python manage.py createsuperuser
```

```
# Run migrations
```

```
python manage.py migrate
```

```
# Generate new migrations (NEVER write manually)
```

```
python manage.py makemigrations
```

```
# Collect static files
```

```
python manage.py collectstatic --noinput
```

```
# Seed the database
```

```
python manage.py seed
```

```
```
```

```

```

```
5. Database Setup & Migrations
```

```
Generate migrations
```

```
```bash
```

```
cd backend
```

```
source .venv/bin/activate
```

```
python manage.py makemigrations
```

```
```
```

```
> Rule: Never manually write migration files. Always use
`makemigrations` to generate them from model changes.
```

```
Apply migrations
```

```
```bash
python manage.py migrate
```
```

### ### Apps and their models

```
App	Models	Purpose
`accounts`	`User` (UUID PK, email, role, first/last name)	User
management with 3 roles: ADMIN, MANAGER, AGENT		
`tickets`	`Ticket` (UUID PK, title, description, priority, status, FK	
to User)	Core ticket management	
`comments`	`Comment` (UUID PK, body, FK to Ticket, FK to User)	
Ticket discussions		
`notifications`	`Notification` (UUID PK, type, is_read, status,	
sent_at, FK to Ticket, FK to User)	In-app notifications + email	
tracking		
`audit`	`TicketHistory` (UUID PK, field_name, old_value, new_value,	
FK to Ticket, FK to User)	Complete change history	
`messaging`	(no models – pure service layer)	RabbitMQ event
publishing |
```

### ### Ticket statuses and priorities

```
```python
class Ticket:
    class Status(models.TextChoices):
        OPEN = "OPEN"
        IN_PROGRESS = "IN_PROGRESS"
        CLOSED = "CLOSED"

    class Priority(models.TextChoices):
        LOW = "LOW"
        MEDIUM = "MEDIUM"
        HIGH = "HIGH"
```
```

```

```

## ## 6. Data Preparation & Seeding

### Seed command

```
```bash
cd backend
source .venv/bin/activate
python manage.py seed
```
```

### Seed command options

| Argument     | Default       | Description                    |
|--------------|---------------|--------------------------------|
| --- --- ---  |               |                                |
| `--users`    | 6             | Number of users to create      |
| `--tickets`  | 25            | Number of tickets to create    |
| `--comments` | 60            | Number of comments to create   |
| `--password` | `testpass123` | Password for all created users |

### What the seed command creates

1. **Users** (11 total by default):

- `admin@example.com` – ADMIN (superuser)
- `manager@example.com` – MANAGER
- `james@example.com` – MANAGER
- 8 agents: `john@example.com`, `mike@example.com`, `lisa@example.com`, `tom@example.com`, `ana@example.com`, `sam@example.com`, `kate@example.com`, `raj@example.com`

2. **Tickets** (25 by default):

- Created via `create\_ticket()` service (triggers notifications + audit)
- Distributed across creators (admin, managers) and assignees (agents)
- Mix of priorities (LOW, MEDIUM, HIGH) and statuses (OPEN, IN\_PROGRESS, CLOSED)

3. **Status changes** on first 12 tickets (triggers notifications + audit history)

4. **Priority changes** on tickets 8-18 (triggers notifications + audit history)

5. **\*\*Reassignment\*\*** on tickets 5-15 (triggers notifications + audit history)

6. **\*\*Comments\*\*** (60 by default):

- Created via `create\_comment()` service (triggers notifications)
- Distributed randomly across tickets and all users

7. **\*\*Notifications\*\*** (~30% marked as read to test badge counts)

### Verify seeded data

```
```bash
```

```
python manage.py shell
```

```
```
```

```
```python
```

```
from django.contrib.auth import get_user_model
```

```
from apps.tickets.models import Ticket
```

```
from apps.comments.models import Comment
```

```
from apps.notifications.models import Notification
```

```
User = get_user_model()
```

```
print(f"Users: {User.objects.count()}")
```

```
print(f"Tickets: {Ticket.objects.count()}")
```

```
print(f"Comments: {Comment.objects.count()}")
```

```
print(f"Notifications: {Notification.objects.count()}")
```

```
```
```

### Login credentials after seeding

```
| Role | Email | Password |
```

```
|---|---|---|
```

```
| Admin | `admin@example.com` | `testpass123` |
```

```
| Manager | `manager@example.com` | `testpass123` |
```

```
| Agent | `john@example.com` | `testpass123` |
```

### Reset and re-seed

```
```bash
python manage.py flush # clears all data
python manage.py migrate # re-apply migrations
python manage.py seed # re-seed
```
```

---

## ## 7. Frontend Setup & Commands

### ### Local setup

```
```bash
cd frontend
npm install
cp .env.example .env # edit if needed
npm run dev
```
```

Frontend available at `http://localhost:5173``.

### ### Build

```
```bash
npm run build
```
```

### ### Lint

```
```bash
npm run lint
```
```

### ### Format

```
```bash
npm run format
```
```

### ### Key frontend technologies

| Package            | Version  | Purpose                 |
|--------------------|----------|-------------------------|
| ---                | ---      | ---                     |
| SvelteKit          | 2        | Framework               |
| Svelte             | 5        | UI library (runes mode) |
| Tailwind CSS       | v4       | Styling                 |
| @tailwindcss/forms | –        | Form element styling    |
| @sentry/sveltekit  | ^10.65.0 | Error monitoring        |
| TypeScript         | –        | Type safety             |

### ### Frontend structure

```

frontend/src/

■■■ lib/

■ ■■■ api/ # HTTP client, error parsing

■ ■■■ components/ # Reusable UI components (Avatar, Badge, Button, Card, etc.)

■ ■■■ stores/ # Global state (auth, notifications)

■ ■■■ types/ # TypeScript interfaces

■ ■■■ constants/ # PRIORITY_COLORS, STATUS_COLORS, etc.

■■■ routes/

■■■ (auth)/ # Login, Register

■■■ (app)/ # Dashboard, Tickets, Notifications, Profile, Settings

```

---

## ## 8. Testing Strategy

### ### Backend tests

```bash

cd backend

source .env/bin/activate

```
# Run all tests
python -m pytest
```

```
# Run with verbose output
python -m pytest -v
```

```
# Run with coverage
python -m pytest --cov=apps --cov-report=term-missing
```

```
# Run specific app tests
python -m pytest apps/accounts/ -v
python -m pytest apps/tickets/ -v
python -m pytest apps/comments/ -v
python -m pytest apps/notifications/ -v
python -m pytest apps/audit/ -v
python -m pytest apps/messaging/ -v
```

```
# Run cache-specific tests
python -m pytest apps/tickets/tests.py::TestDashboardCache -v
```

```
# Run Celery task tests (eager mode)
python -m pytest apps/notifications/tests.py -v
...
```

```
### Test count breakdown
```

| App | Tests | Focus |
|---------------|---------|--------------------------------------|
| --- | --- | --- |
| accounts | 20 | Authentication, roles, JWT |
| tickets | 42 | CRUD, permissions, cache, dashboard |
| comments | 25 | CRUD, permissions |
| notifications | 44 | Service, Celery tasks, email, status |
| audit | 14 | History tracking |
| messaging | 12 | RabbitMQ publishing (mocked) |
| **Total** | **157** | |

```
### Celery testing approach
```

Tests use `CELERY\_TASK\_ALWAYS\_EAGER=True` to execute tasks synchronously:

```
```python
backend/conftest.py (set BEFORE Django import)
os.environ.setdefault("CELERY_TASK_ALWAYS_EAGER", "True")
os.environ.setdefault("CELERY_TASK_EAGER_PROPAGATES", "True")
```
```

Frontend tests

```
```bash
cd frontend
npm test
```
```

Linting

```
```bash
Backend
ruff check .
ruff format --check .
```

```
Frontend
npm run lint
```
```

9. API Endpoints Reference

Authentication

| Method | Endpoint | Description |
|--------|----------------------------|------------------------------------|
| POST | `/api/auth/register/` | Register a new user |
| POST | `/api/auth/login/` | Obtain JWT access + refresh tokens |
| POST | `/api/auth/token/refresh/` | Refresh access token |
| POST | `/api/auth/logout/` | Blacklist refresh token |

| | | |
|-----------|-----------------|-----------------------------|
| GET | `/api/auth/me/` | Get current user profile |
| PUT/PATCH | `/api/auth/me/` | Update current user profile |

Tickets

| | | |
|-----------|---------------------------------|---|
| Method | Endpoint | Description |
| --- | --- | --- |
| GET | `/api/tickets/` | List tickets (filter, search, paginate) |
| POST | `/api/tickets/` | Create a ticket |
| GET | `/api/tickets/{id}/` | Get ticket detail |
| PUT/PATCH | `/api/tickets/{id}/` | Update a ticket |
| DELETE | `/api/tickets/{id}/` | Delete a ticket |
| POST | `/api/tickets/{id}/assign/` | Assign a ticket |
| GET | `/api/tickets/dashboard_stats/` | Aggregate statistics |
| GET | `/api/tickets/my_stats/` | Per-user statistics |

Comments

| | | |
|-----------|------------------------------|----------------------------|
| Method | Endpoint | Description |
| --- | --- | --- |
| GET | `/api/comments/?ticket={id}` | List comments for a ticket |
| POST | `/api/comments/` | Create a comment |
| PUT/PATCH | `/api/comments/{id}/` | Update a comment |
| DELETE | `/api/comments/{id}/` | Delete a comment |

Notifications

| | | |
|--------|------------------------------------|--|
| Method | Endpoint | Description |
| --- | --- | --- |
| GET | `/api/notifications/` | List notifications (filter by read/unread) |
| POST | `/api/notifications/{id}/read/` | Mark as read |
| GET | `/api/notifications/unread_count/` | Get unread count |

Documentation

| | |
|----------|-------------|
| Endpoint | Description |
| --- | --- |

```
`/api/docs/`	Swagger UI
`/api/docs/redoc/`	ReDoc
`/api/docs/schema/`	OpenAPI schema (JSON)
`/admin/`	Django admin
`/sentry-debug/`	Sentry test endpoint
```

10. Environment Variables

Backend (`backend/.env`)

```
```env
```

```
Django
```

```
SECRET_KEY=your-secret-key
```

```
DEBUG=True
```

```
ALLOWED_HOSTS=localhost,127.0.0.1
```

```
ENVIRONMENT=development
```

```
PostgreSQL
```

```
DB_NAME=ticket_db
```

```
DB_USER=ticket_user
```

```
DB_PASSWORD=ticket_pass
```

```
DB_HOST=localhost
```

```
DB_PORT=5432
```

```
Celery (Redis)
```

```
CELERY_BROKER_URL=redis://localhost:6379/0
```

```
CELERY_RESULT_BACKEND=redis://localhost:6379/0
```

```
Redis cache
```

```
REDIS_URL=redis://localhost:6379/1
```

```
RabbitMQ
```

```
RABBITMQ_HOST=localhost
```

```
RABBITMQ_PORT=5672
```

```
RABBITMQ_USERNAME=guest
```

```
RABBITMQ_PASSWORD=guest
```

```
RABBITMQ_VHOST=
```

```
Email (Gmail SMTP)
```

```
EMAIL_BACKEND=django.core.mail.backends.smtp.EmailBackend
```

```
EMAIL_HOST=smtp.gmail.com
```

```
EMAIL_PORT=587
```

```
EMAIL_USE_TLS=True
```

```
EMAIL_HOST_USER=your_email@gmail.com
```

```
EMAIL_HOST_PASSWORD=your_google_app_password
```

```
DEFAULT_FROM_EMAIL=your_email@gmail.com
```

```
CORS
```

```
CORS_ALLOWED_ORIGINS=http://localhost:5173
```

```
Sentry (leave empty to disable)
```

```
SENTRY_DSN=
```

```
SENTRY_TRACES_SAMPLE_RATE=0.1
```

```
```
```

```
### Frontend (`frontend/.env`)
```

```
```env
```

```
PUBLIC_API_BASE_URL=http://localhost:8000/api
```

```
PUBLIC_SENTRY_DSN=
```

```
PUBLIC_SENTRY_ENVIRONMENT=development
```

```
```
```

```
---
```

```
## 11. Key Technical Decisions
```

```
### Celery Integration
```

****Problem:**** Sending emails synchronously blocked the request for 1-5 seconds and caused request failures on email errors.

****Solution:**** Celery task `send\_notification\_email` with:

- `autoretry\_for=(Exception,)` – auto-retry on failure

- ``retry_backoff=True`` – exponential backoff
- ``max_retries=3`` – hard limit
- Task receives only primitive values (no model instances)
- Separate email builder (``email.py``) from task logic

Refresh Token Rotation

****Problem:**** Frontend discarded the new refresh token from the backend response, causing forced re-login after one refresh.

****Solution:**** Backend has ``ROTATE_REFRESH_TOKENS=True`` and ``BLACKLIST_AFTER_ROTATION=True``. Frontend's ``doRefresh()`` now stores the new refresh token when the backend returns one.

Redis Cache Invalidation

****Problem:**** ``cache.clear()`` is dangerous and can cause data inconsistencies.

****Solution:**** Only targeted key deletion. ``_invalidate_ticket_caches()`` deletes specific ``dashboard_stats`` and ``my_stats_{user_id}`` keys when tickets are created, updated, deleted, or reassigned.

RabbitMQ vs Redis for Events

****Decision:**** Use Redis for Celery task queuing (simpler, battle-tested), and RabbitMQ for domain event publishing (better topic routing, independent of task processing).

Sentry

****Problem:**** Need error monitoring without performance overhead when not configured.

****Solution:**** Backend guards Sentry init with ``if SENTRY_DSN:``. Frontend uses ``$app/environment`` to only initialize in browser. When DSN is empty, zero overhead.

UI Design System

****Problem:**** Inconsistent styling across pages with ad-hoc color classes.

****Solution:**** Unified design system using CSS `@theme` variables in `layout.css`:

- `primary-` tokens for interactive/accent elements (indigo)
- `surface-` tokens for neutral text, borders, backgrounds
- Utility classes: `.glass`, `.gradient-primary`, `.card-hover`, `.stat-card`
- All components updated to use consistent color scheme

Makefile Quick Reference

Run `make help` to see all commands:

````bash`

```
make up # Start all Docker services
make down # Stop all services
make build # Rebuild Docker images
make logs # Tail all logs
make migrate # Apply database migrations
make seed # Seed database with sample data
make createsuperuser # Create Django admin user
make test-backend # Run backend tests with coverage
make lint-backend # Lint backend with ruff
make format-backend # Auto-format backend with ruff
make lint-frontend # Lint frontend
make dev-backend # Run Django dev server (local)
make dev-frontend # Run SvelteKit dev server (local)
make celery-worker # Run Celery worker (local)
```

`````